Fire and Smoke Image Classification Using Adaptive Digital Image Processing and a Lightweight Vision Transformer

Mahdia Hossain
Department of Computer Science and Engineering
University of Liberal Arts Bangladesh (ULAB)
Dhaka, Bangladesh
CSE 4462 Digital Image Processing Lab, Open-Ended Lab

***Abstract*—Fires cause great damage when they are found late. Cameras can help, but camera photos are often noisy, and many normal things look like fire or smoke. This paper presents a system that looks at one photo and says fire, smoke or normal. The system first measures the noise in the photo with the Immerkær noise estimate. It applies a 3×3 median filter and CLAHE on the lightness channel only when the noise level σ is above 2.56. Clean photos are not changed. A small vision transformer, MobileViT-XS, then classifies the photo. The model is trained with added Gaussian noise. We built a clean dataset of 41,789 photos from three public sources and a separate set of 451 look-alike photos such as sunsets, lamps and fog. We trained 16 classifiers and chose the final one by validation score only. The final model reaches a macro-F1 of 0.917 on validation, 0.895 on the in-domain test set and 0.900 on a test set from a new source. It gives false alarms on only 2.2% of the look-alike photos. Under added noise, blur, darkness and JPEG damage its macro-F1 stays at 0.81 or higher, while the same network trained on raw photos falls to 0.40. The system is released as a small Python package that shows every intermediate step.**

***Index Terms*—fire detection, smoke detection, digital image processing, noise estimation, median filter, CLAHE, MobileViT, robustness**

I. Introduction

Fires in homes, factories, streets and forests cause great damage when they are found late. Cameras are already in many places. A program that checks camera photos can raise an early alarm [1].

The task in this work is simple to state. The input is one colour photo. The output is one of three classes: fire, smoke or normal. The task is hard for three reasons. First, many normal things look like fire or smoke. Sunsets, lamps and car lights look like fire. Fog and steam look like smoke. Second, real camera photos are often noisy, dark, blurred or heavily compressed. Deep networks lose accuracy on such photos [2], [3]. Third, a model must also work on photos from places it did not see in training.

Digital image processing (DIP) gives classic tools for these problems. A median filter removes noise dots [4]. CLAHE improves local contrast [5]. But a filter also removes some real detail. If we filter every photo, clean photos lose detail for no reason.

Our idea is to filter a photo only when it needs it. The system measures the noise level of each photo with the Immerkær estimate [6]. If the noise is above a threshold, the photo gets a median filter and CLAHE. If not, the photo is left unchanged. A small vision transformer, MobileViT-XS [7], then classifies the photo. We call this final model CA. Fig. 1 shows the full pipeline.

The main contributions of this work are:

- A clean dataset of 41,789 photos built from three public sources, with copies and test leaks removed, and a hand-checked set of 451 look-alike photos.
- A step-by-step study of the DIP steps (filters, contrast methods, colour segmentation, morphology and noise estimation), measured with PSNR, SSIM and other scores.
- An adaptive DIP preprocessing rule that filters only noisy photos, combined with noise training of MobileViT-XS.
- A fair comparison of 16 classifiers, chosen by validation score only, and a robustness test with nine kinds of image damage.
- A small Python package, firesmoke, that runs the model on a new photo and saves every intermediate step.

II. Related Work

A. Colour rules for fire

Early fire detection methods used colour rules. Chen et al. [8] used rules on the red, green and blue values and on saturation. Töreyin et al. [9] added motion and flicker cues for video. Çelik and Demirel [10] gave a rule in the YCbCr colour space, which separates brightness from colour. We use their rule for our fire mask. Colour rules are fast and easy to explain, but they give false alarms on anything red or bright.

B. Deep learning for fire and smoke

Later work used convolutional neural networks (CNNs). Muhammad et al. [1] used a CNN for early fire detection in surveillance cameras. De Venâncio et al. [11] built the D-Fire dataset and a detector for low-power devices. Catargiu et al. [12] built the FireAndSmoke dataset and compared YOLO detectors on it. That dataset also has hard normal photos that can be mistaken for fire.

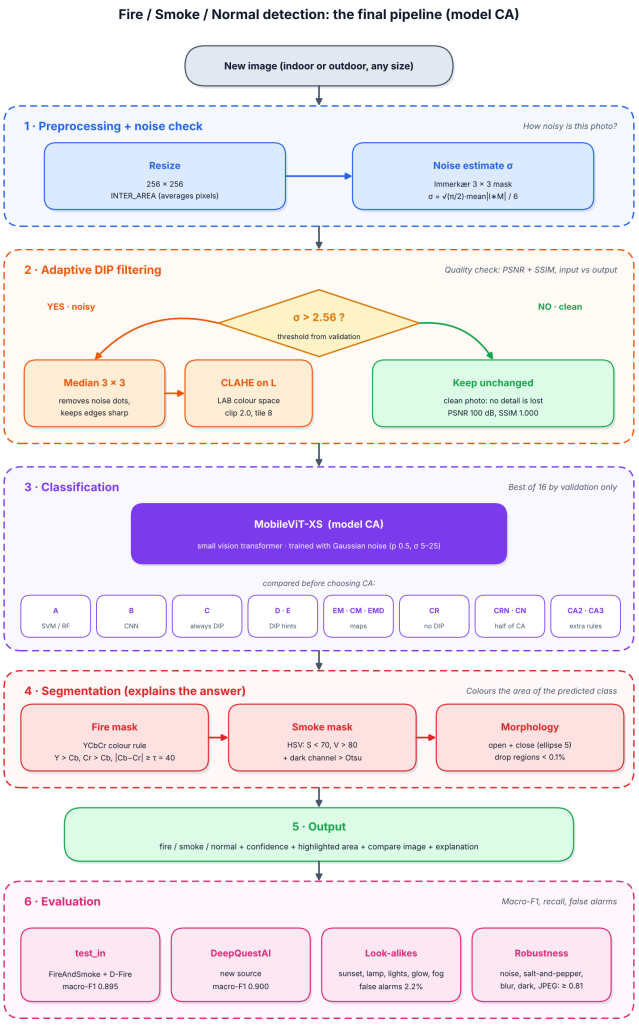


Fig. 1. The final pipeline (model CA). A photo is filtered only when its measured noise σ is above 2.56. The colour masks are used only to explain the answer.

Small networks such as MobileNetV3 [13] and MobileViT [7] can run on small devices.

C. Image quality and deep networks

Dodge and Karam [2] showed that noise and blur reduce the accuracy of deep networks. Hendrycks and Dietterich [3] made a standard test with many kinds of image damage. Our robustness test follows the same idea. Our work joins the two lines: classic DIP is used to clean the photo, and a small transformer is used to classify it.

III. Dataset

A. Sources

We used three public datasets. Table I lists them. FireAndSmoke [12] and D-Fire [11] were used for training, validation and in-domain testing. Both have boxes drawn around fire and smoke. A photo with a fire box is labelled fire. A photo with only smoke boxes is labelled smoke. A photo with no box is labelled normal. The DeepQuestAI dataset [14] was kept only for testing. It comes from a different source, so it shows how the model works on new places and cameras.

TABLE I
Datasets used

Dataset	Photos scanned	Use
FireAndSmoke [12]	27,294	train, validation, test
D-Fire [11]	21,527	train, validation, test, mask check
DeepQuestAI [14]	3,000	test only (new source)

B. Cleaning

The same photo often appears more than once on the internet. If a copy of a test photo is in the training set, the test score becomes too high. We removed 157 exact copies with a file hash and 9,875 near-copies with a perceptual hash (distance of 4 or less). We also removed 8,119 training and validation photos that were copies of a test photo. Fig. 2 shows examples. The final dataset has 41,789 photos.

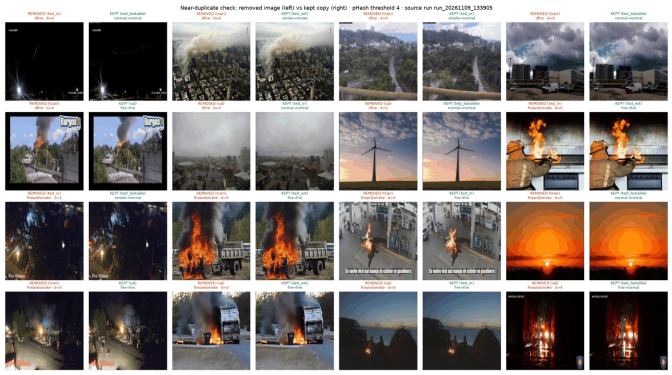


Fig. 2. Examples of near-copy pairs found by the duplicate check. One photo of each pair was removed.

C. Splits

Table II shows the splits. The validation set was used to choose thresholds and the best model. The test sets were never used for any choice.

TABLE II
PHOTOS PER SPLIT AFTER CLEANING

Split	Fire	Smoke	Normal	Total
Train	12,837	6,239	4,971	24,047
Validation	4,805	1,922	598	7,325
Test in-domain	2,858	1,930	2,022	6,810
Test DeepQuestAI	940	993	998	2,931
Look-alike pool	0	0	676	676

D. Look-alike set

We made a separate set of normal photos that look like fire or smoke. The pool had 676 photos. We checked them by hand in a small labelling tool. We kept 451 photos in five types: sunset or sunrise (88), lamp or street light (52), vehicle light (185), reflection or glow (51) and cloud, fog or haze (75). We rejected 19 photos that really showed fire or smoke. The other photos were skipped or left unlabelled and were not used. Fig. 3 shows one example row per type. A good model must call all of these photos normal.

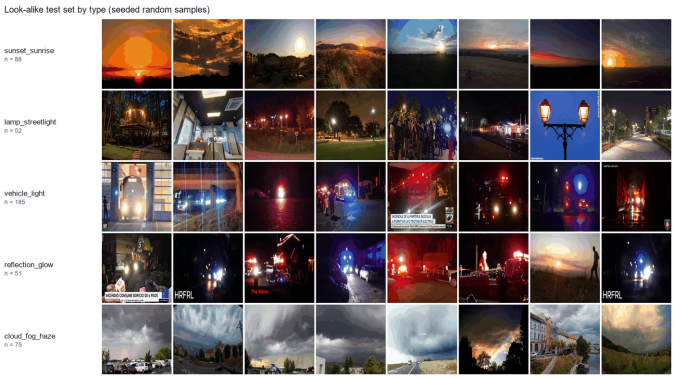


Fig. 3. Examples of each look-alike type. A good model must call all of these normal.

IV. METHOD

A. Overview

Fig. 1 shows the pipeline. It has four parts: resizing, noise measurement, adaptive filtering and classification. A fifth part, colour segmentation, is used only to show the user where the fire or smoke is.

B. Resizing

Each photo is resized to 256×256 pixels with area interpolation. Area interpolation averages blocks of pixels. It works like a low-pass filter, so it also removes a little noise [15].

C. Noise measurement

We measure the noise with the method of Immerkær [6]. The grey image I is convolved with the 3×3 mask M:

$$M = \begin{bmatrix} 1 & -2 & 1 \\ -2 & 4 & -2 \\ 1 & -2 & 1 \end{bmatrix} \tag{1}$$

The mask is the difference of two Laplacian masks. Its response is zero on flat areas and on straight edges. So what is left is mostly noise. The noise level σ is then

$$\sigma = \sqrt{\frac{\pi}{2}} \, \frac{1}{6(W-2)(H-2)} \sum_{x,y} |\,(I * M)(x,y)\,| \tag{2}$$

Here W and H are the width and height of the image. The method needs only one convolution, so it is very fast.

D. Adaptive filtering

If σ is above 2.56, the photo is called noisy. A noisy photo gets two steps. First, a 3×3 median filter replaces each pixel with the middle value of its neighbours [4]. A single wrong dot is never the middle value, so it is removed, and edges stay sharp. Second, CLAHE [5] is applied to the L channel of the LAB colour space with a clip limit of 2.0 and 8×8 tiles. CLAHE improves local contrast. Working on L only keeps the colours unchanged. If σ is 2.56 or below, the photo is passed on unchanged. The threshold 2.56 was chosen on validation photos (Section V-E).

E. Classifier and training

The classifier is MobileViT-XS [7] from the timm library [16], pretrained on ImageNet. It has 1.93 million parameters. We trained it in PyTorch [17] for 15 epochs with the AdamW optimiser [18]. Table III lists the settings. During training, half of the photos get extra Gaussian noise with a random σ between 5 and 25. The noise is added before the adaptive step, so the model sees filtered noisy photos during training. This teaches the model to ignore the noise that the filter leaves behind. The epoch with the best validation macro-F1 is kept.

TABLE III
TRAINING SETTINGS

Setting	Value
Input size	256 × 256
Batch size	32
Epochs	15
Optimiser	AdamW, weight decay 0.05
Learning rate	0.0005, cosine schedule, 1 warm-up epoch
Loss	cross-entropy, class weights, label smoothing 0.1
Noise training	Gaussian, probability 0.5, σ from 5 to 25
Random seed	42
Hardware	NVIDIA GeForce RTX 3050 (6 GB) laptop

F. Segmentation for explanation

After the model gives its answer, a colour mask shows the area of the predicted class. The fire mask uses the YCbCr rule of Çelik and Demirel [10]. A pixel is fire if $Y > Cb$, $Cr > Cb$ and $|Cb - Cr| \geq \tau$, with $\tau = 40$. The smoke mask marks pixels with low saturation and enough brightness ($S < 70$ and $V > 80$ in HSV) whose dark channel [19] is above an Otsu threshold [20]. Both masks are cleaned with morphological opening and closing with a 5×5 ellipse, and regions smaller than 0.1% of the image are removed [15]. The masks do not change the answer of the model.

G. Hand-made features

For the baseline models we also turned each photo into 16 numbers: 4 colour features, 3 brightness features, 4 texture features from local binary patterns [21] and the grey-level co-occurrence matrix [22], 2 haze features from the dark channel and 3 edge features.

H. Models compared

We trained 16 classifiers. Model A uses only the 16 features with an SVM [23] or a Random Forest [24]. Model B is a MobileNetV3-Small CNN [13]. All other models use MobileViT-XS. Model C always applies the median filter and CLAHE. Model D joins the 16 features to the output of the network. Model E gives the 16 features to the transformer as extra tokens. Models EM, CM and EMD add an extra branch that predicts fire and smoke maps. Model CR uses raw photos with no DIP. CRN is CR with noise training. CN is C with noise training. CA is the adaptive model. CA2 adds a brightness rule for dark photos. CA3 picks a different filter for each noise type.

I. Measures

The main score is macro-F1, the average F1 score of the three classes. We also report recall and the false alarm rate, which is the share of normal photos called fire or smoke. For CA we report 95% confidence intervals from 1,000 bootstrap samples [25]. Image change is measured with PSNR and SSIM [26]:

$$\mathrm{PSNR} = 10 \log_{10} \left(\frac{255^2}{\mathrm{MSE}} \right) \tag{3}$$

MSE is the mean squared difference between two images. A higher PSNR or SSIM means that the two images are more alike.

V. DIP EXPERIMENTS

A. How much each step changes a photo

We applied each preprocessing step to 1,300 photos and compared the output with the input. Table IV shows the result. The median filter changes a photo only a little. CLAHE changes it more, because it spreads the brightness values. Entropy rose from 7.05 to 7.33 bits and RMS contrast rose from 53.8 to 56.2. Fig. 4 shows each step on example photos, and Fig. 5 shows the histogram before and after CLAHE.

TABLE IV
CHANGE CAUSED BY EACH STEP (1,300 PHOTOS, OUTPUT COMPARED WITH INPUT)

Step	PSNR (dB)	SSIM
Median 3 × 3	32.32	0.925
CLAHE on L	21.07	0.799
Both together	20.83	0.786



Fig. 4. Each preprocessing step on example photos: original, resized, median filter, CLAHE on L.

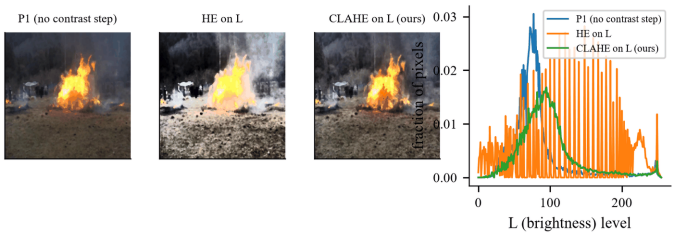


Fig. 5. The brightness histogram spreads out after CLAHE.

B. Comparing filters and contrast methods

We added noise to photos on purpose and tested how well each filter brings the clean photo back. Table V shows the result. On salt-and-pepper noise the median filter is clearly the best. On Gaussian noise the bilateral filter [27] is the best. Fig. 6 shows an example.

TABLE V
RESTORING A NOISY PHOTO (PSNR IN DB / SSIM, COMPARED WITH THE CLEAN PHOTO)

Noise added	Median	Gaussian	Bilateral
Salt-and-pepper	32.15 / 0.923	27.87 / 0.750	21.67 / 0.622
Gaussian	29.97 / 0.819	31.10 / 0.848	34.56 / 0.894

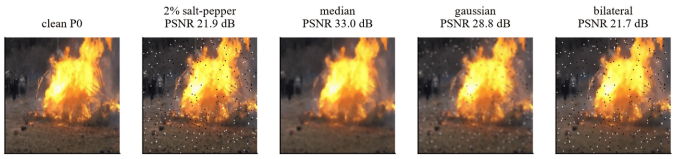


Fig. 6. Median, Gaussian and bilateral filters compared on a noisy photo.

We also compared three contrast methods by how much they shift the colours. Histogram equalisation on the RGB channels shifted colours by 18.4. Histogram equalisation on L shifted them by 1.28. CLAHE on L shifted them by only 0.32. Colour is the strongest sign of fire, so we chose CLAHE on L. Fig. 7 shows the difference.

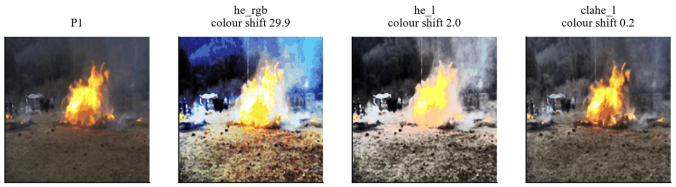


Fig. 7. Histogram equalisation on RGB changes colours a lot. CLAHE on L keeps the fire colour correct.

C. Segmentation

We checked the colour masks against the boxes of the datasets. Fig. 8 shows the steps and Fig. 10 shows more examples. We tested five values of the fire threshold τ on validation photos. The value $\tau = 40$ was the best (Fig. 9). Table VI gives the result. The fire mask works fairly well. The smoke mask is weak. It marks almost 19% of the area of normal photos, because grey sky, roads and walls have the same colour as smoke. For this reason the masks are used only as features for the baseline models and as a picture for the user.

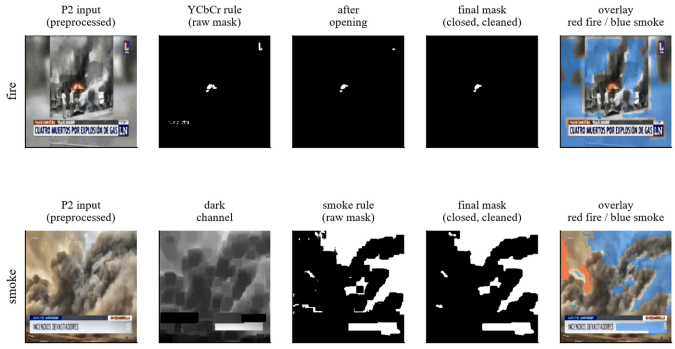


Fig. 8. Segmentation step by step. Top: fire (YCbCr rule, opening, closing, overlay). Bottom: smoke (dark channel, smoke rule, cleaned mask, overlay).

TABLE VI
COLOUR MASKS CHECKED ON VALIDATION PHOTOS

Mask	Covers box area	Area marked on normal photos	Score J
Fire ($\tau = 40$)	33%	3.65%	0.294
Smoke	33.7%	18.9%	0.147

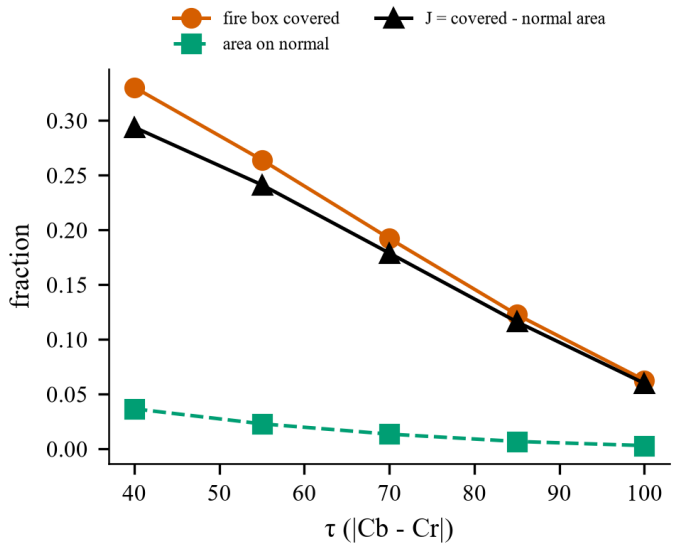


Fig. 9. Choosing the fire threshold τ on validation photos. $\tau = 40$ was the best.



Fig. 10. More mask examples. The smoke mask often also marks grey sky, roads and walls.

D. Features

The 16 features were computed for 41,564 photos with no failures. This took about 22 ms per photo. Fig. 11 ranks the features by the ANOVA F-score on training data. The strongest features are the edges of the fire mask (2568), redness (1452), fire area (1021) and the mean of the dark channel (999).

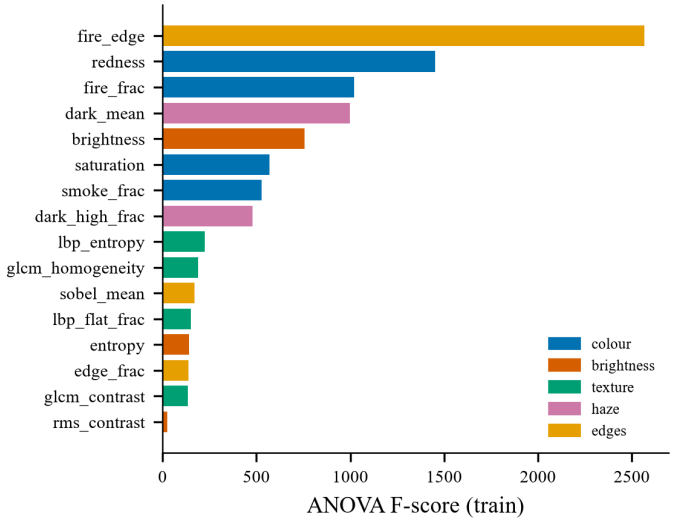


Fig. 11. The 16 features ranked by ANOVA F-score on training data.

E. Noise threshold

To choose the noise threshold we measured σ on original validation photos and on the same photos with added Gaussian noise. Fig. 12 shows the two groups. The threshold $\sigma = 2.56$ separates them best, with a balanced accuracy of 0.784. It marks 97% of the photos with added noise of σ 15 and all photos with σ 25. It also marks 36% of the original photos. This is partly correct, because many camera photos already have some noise.

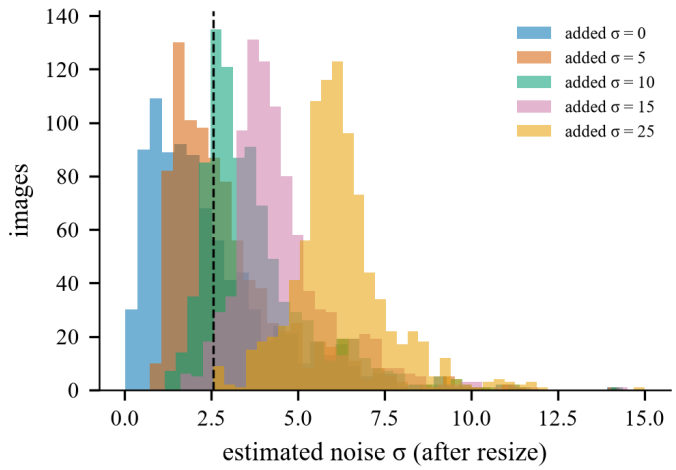


Fig. 12. Noise level σ of original photos and photos with added noise. The chosen threshold is 2.56.

F. Brightness threshold

We also tested a rule for dark photos. A mean brightness of 71.4 separates normal photos from darkened photos with a balanced accuracy of 0.87 (Fig. 13). But it marks 21.9% of ordinary photos as dark. Model CA2 used this rule. Its validation score was lower than that of CA (0.908 against 0.917), and it gave more false alarms on look-alike photos (3.1% against 2.2%), so the rule is not in the final model.

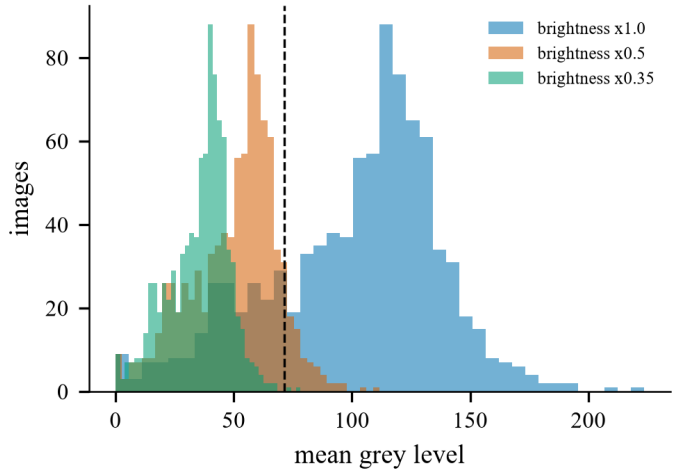


Fig. 13. Mean brightness of ordinary and darkened photos. The tested threshold is 71.4.

G. Which filter for which noise

In a larger study we added noise to 150 validation photos and compared 15 filter settings and a no-filter baseline. The filters included median, Gaussian, average, bilateral and non-local means [28] filters. The first run of this study added the noise after resizing. This was a mistake, because a real camera adds noise before the photo is resized. We corrected the study and report the corrected run. Table VII shows the best filter for each noise type. Fig. 14 shows examples and Fig. 15 shows all SSIM values.

TABLE VII
BEST FILTER FOR EACH NOISE TYPE (NOISE ADDED BEFORE RESIZING)

Noise added	Best filter	PSNR (dB)	SSIM
Gaussian σ 10	Bilateral	37.1	0.932
Gaussian σ 25	Bilateral	33.5	0.872
Salt-and-pepper 2%	Median 3 $\times$ 3	31.5	0.903
Salt-and-pepper 5%	Median 3 $\times$ 3	30.1	0.844



Fig. 14. The same noisy photos restored by different filters.

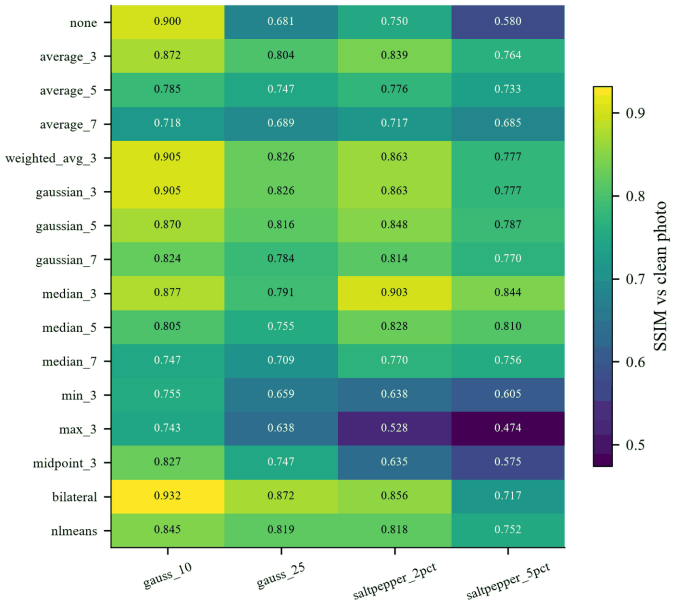


Fig. 15. SSIM of each filter for each noise type.

We also tested how well two detectors find noisy photos (Fig. 16). The noise estimate σ finds both noise types, with an AUC of 0.839 for Gaussian noise and 0.964 for salt-and-pepper noise. The isolated point count (a mask with 8 in the centre and -1 around it, which marks lonely dots) finds salt-and-pepper noise almost perfectly when the noise is added after resizing (AUC 0.999), but poorly when it is added before (AUC 0.621). Resizing shrinks the dots into small grey spots, so they are no longer lonely points.

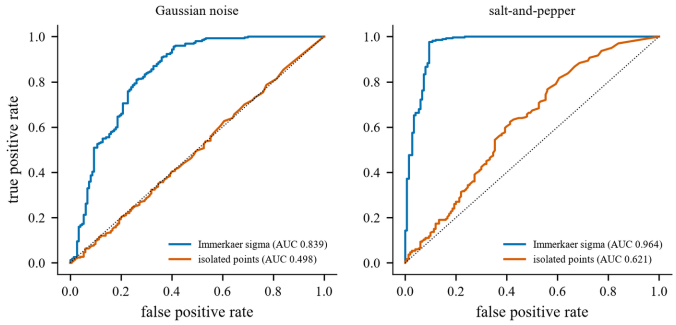


Fig. 16. How well the noise estimate σ and the isolated point count find noisy photos.

Fig. 17 shows the adaptive rule on example photos. Clean photos pass unchanged, with a PSNR of 100 dB and an SSIM of 1.000. Photos with added noise are filtered.

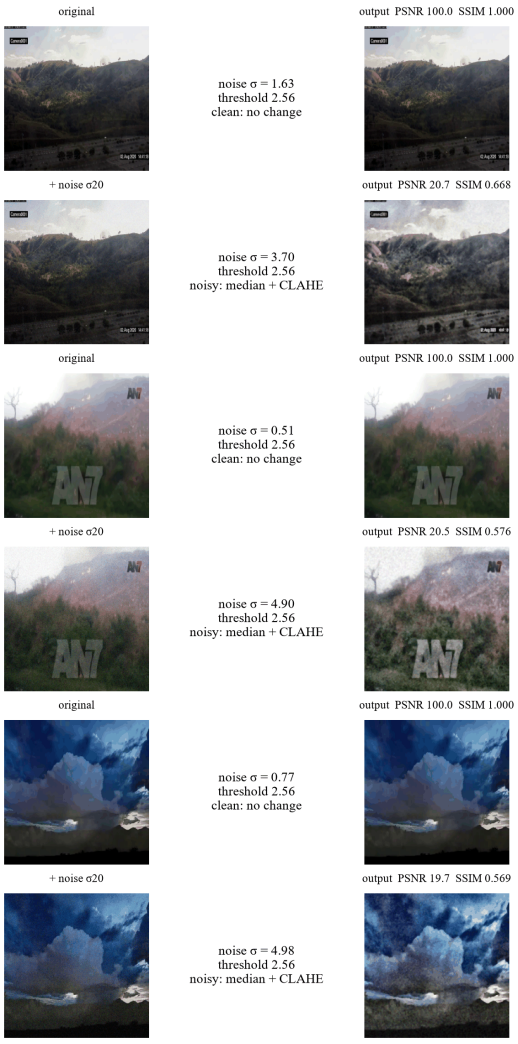


Fig. 17. The adaptive rule in action. Clean photos pass unchanged. The same photos with added noise get the median filter and CLAHE.

VI. Classification Results

Table VIII gives the scores of all 16 classifiers. Fig. 18 shows the validation scores and Fig. 19 shows the false alarms on look-alike photos. Each model was trained once with seed 42.

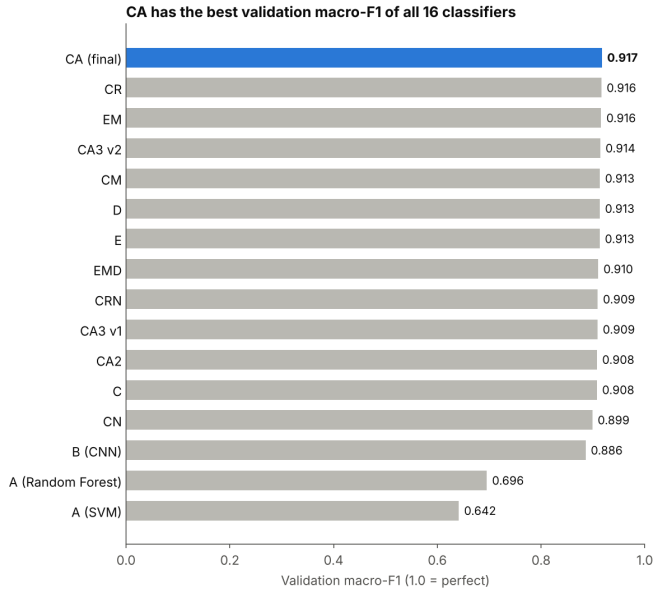


Fig. 18. Validation macro-F1 of all 16 classifiers. Only this score was used to choose the final model.

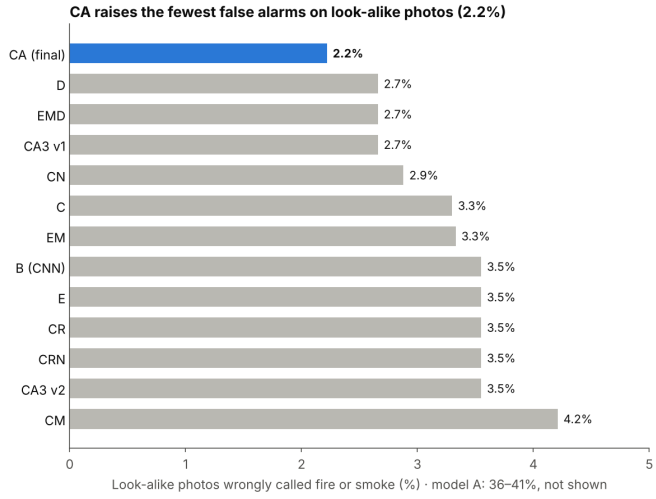


Fig. 19. Share of look-alike photos wrongly called fire or smoke. Model A (36% to 41%) is not shown.

A. Baselines

Model A reaches only 0.70 on validation and about 0.51 on DeepQuestAI. It calls 77% to 82% of the normal DeepQuestAI photos fire or smoke (Fig. 20). Sixteen numbers cannot describe a photo well enough. Model B, the CNN, reaches 0.886 on validation. Every MobileViT-XS model is better than B.

TABLE VIII

ALL 16 CLASSIFIERS. SCORES ARE MACRO-F1. FALSE ALARMS ARE THE SHARE OF NORMAL PHOTOS CALLED FIRE OR SMOKE.

Model	Input and preprocessing	Validation	Test in-domain	Test DeepQuestAI	False alarms DeepQuestAI (%)	False alarms look-alikes (%)
CA (final)	**adaptive DIP + noise training**	**0.9174**	**0.8950**	**0.9002**	**12.4**	**2.22**
CR	raw photos, no DIP	0.9164	0.9017	0.8995	12.0	3.55
EM	feature tokens + learned maps	0.9158	0.8888	0.8884	17.4	3.33
CA3 v2	filter chosen per noise type (corrected)	0.9142	0.8910	0.8912	15.7	3.55
CM	learned maps	0.9133	0.8944	0.8753	16.5	4.21
D	16 features joined at the end	0.9132	0.8912	0.8981	10.7	2.66
E	16 features as tokens	0.9130	0.8945	0.8781	18.7	3.55
EMD	feature tokens + maps from DIP masks	0.9102	0.8871	0.8827	13.1	2.66
CRN	raw photos + noise training	0.9092	0.8905	0.8974	13.2	3.55
CA3 v1	filter chosen per noise type (first run)	0.9089	0.8883	0.9044	13.1	2.66
CA2	adaptive DIP + brightness rule	0.9078	0.8901	0.9010	10.5	3.10
C	always median + CLAHE	0.9077	0.8897	0.8833	15.5	3.30
CN	always median + CLAHE + noise training	0.8991	0.8868	0.8871	16.8	2.88
B	MobileNetV3-Small CNN	0.8859	0.8776	0.8555	22.2	3.55
A (RF)	16 features, Random Forest	0.6955	0.7105	0.5078	81.6	40.8
A (SVM)	16 features, SVM	0.6416	0.6896	0.5243	76.8	35.7

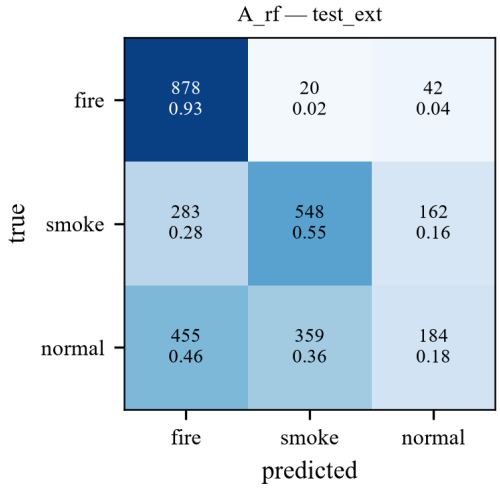


Fig. 20. Confusion matrix of model A (Random Forest) on DeepQuestAI. Most normal photos are called fire or smoke.

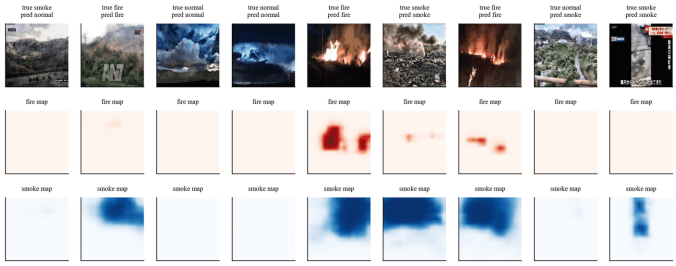


Fig. 21. Fire and smoke maps learned by model EM.

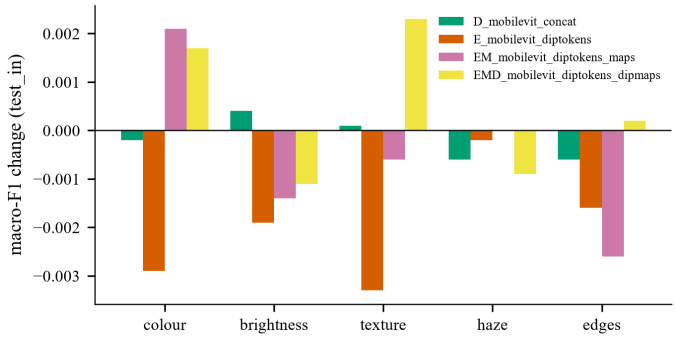


Fig. 22. Change in macro-F1 on the in-domain test set when one group of features is switched off at test time.

B. Models with DIP features inside the network

Models D, E, EM, CM and EMD give DIP information to the network in different ways. Fig. 21 shows maps learned by model EM. Their validation scores are between 0.910 and 0.916. None of them is better than CA. To see if these models really use the features, we switched off one group of features at test time (Fig. 22). For D and EM the macro-F1 changed by less than 0.5 points. Only model E lost a little on DeepQuestAI (0.013 for texture and 0.011 for edges). So the transformer learns almost everything from the photo itself. After this result we stopped adding features and worked on cleaning the photo instead.

C. Raw photos against DIP

Model CR uses raw photos. On clean photos it is very good. It has the best in-domain test score (0.902). Model C, which filters every photo, is a little lower on all three sets. This shows that filtering clean photos does not help. But the robustness test in Section VI-E shows that CR breaks down on noisy photos.

D. The final model

Model CA has the best validation macro-F1 of all 16 classifiers (0.9174), so it was chosen as the final model. Table IX gives its scores with 95% confidence intervals. Fig. 23 shows the training curve and Fig. 24 shows the confusion matrix on DeepQuestAI. On the in-domain test set, the fire recall is 0.955 and the smoke recall is 0.815. CA is not the best on every single test. CR is higher on the in-domain test, CA3 v1 and CA2 are a little higher on DeepQuestAI, and CA2 and D have fewer false alarms on DeepQuestAI. The macro-F1 differences are small and lie inside the confidence intervals of CA.

TABLE IX
FINAL MODEL CA WITH 95% CONFIDENCE INTERVALS

Split	Photos	Accuracy	Macro-F1	Macro-F1 95% interval	False alarms
Validation	7,325	0.942	0.917	0.9085 to 0.9257	5.5%
Test in-domain	6,810	0.904	0.895	0.8877 to 0.9024	8.3%
Test DeepQuestAI	2,931	0.900	0.900	0.8898 to 0.9116	12.4%
Look-alikes	451	n/a	n/a	n/a	2.2%

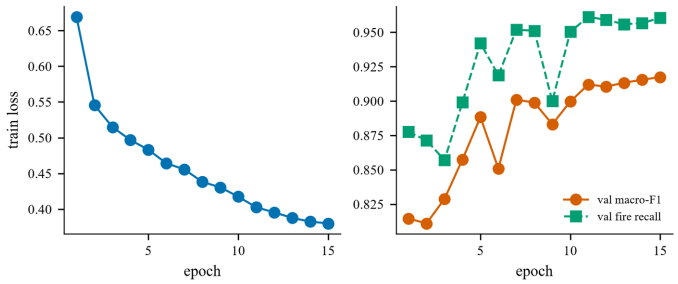


Fig. 23. Training curve of model CA. The best epoch was the last one, epoch 15.

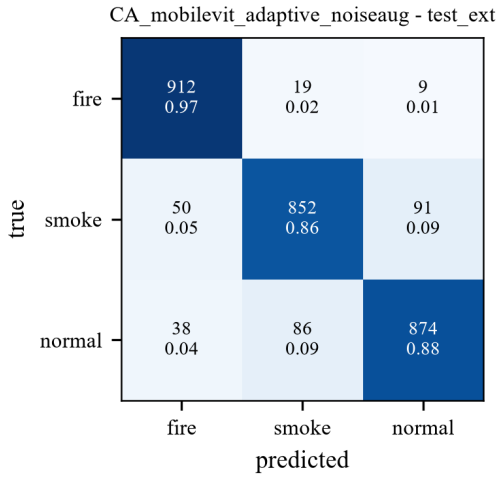


Fig. 24. Confusion matrix of model CA on DeepQuestAI.

Table X shows the false alarms of CA on each look-alike type. Only 10 of the 451 photos were called fire or smoke. No fog photo was called smoke.

TABLE X
FALSE ALARMS OF MODEL CA ON THE LOOK-ALIKE SET

Type	Photos	Called fire	Called smoke	False alarm rate
Cloud, fog, haze	75	0	0	0.00%
Lamp, street light	52	2	0	3.85%
Reflection, glow	51	1	1	3.92%
Sunset, sunrise	88	0	1	1.14%
Vehicle light	185	0	5	2.70%
All	451	3	7	2.22%

E. Robustness to damaged photos

We damaged the DeepQuestAI photos on purpose and measured macro-F1 again. Nothing was retrained. The damage was added to the full-size photo, before the pipeline. We used nine kinds of damage: darkening (brightness times 0.35), Gaussian blur (7×7 kernel), JPEG compression (quality 20), Gaussian noise with σ of 5, 10, 15 and 25, and salt-and-pepper noise on 2% and 5% of the pixels. Table XI and Fig. 25 show the result.

On clean, dark, blurred and JPEG photos the six models are close. The difference appears under noise. CR falls to 0.40 at Gaussian noise of σ 25 and to 0.43 at 5% salt-and-pepper noise. Its fire recall at σ 25 is only 0.38, so it misses most fires. CRN has noise training but no filter. It handles Gaussian noise but falls to 0.44 at 5% salt-and-pepper noise. CN has both the filter and noise training, but it filters clean photos too, and its validation score is the lowest of the MobileViT-XS models. CA and CA2 both stay at 0.81 or higher under every kind of damage. The fire recall at σ 25 is 0.91 for CA and 0.906 for CA2. CA2, which also brightens dark photos, is a little better than CA on dark photos (0.877 against 0.868), on JPEG photos and on strong Gaussian noise (0.837 against 0.815 at σ 25). The two are almost equal on salt-and-pepper noise. CA2 was still not chosen, because its validation score is lower and it gives more false alarms on look-alike photos. The final model was chosen by validation score only, before these tests were run. An earlier test of model C, which has the filter but no noise training, gave 0.546 at σ 25. So both parts are needed: the filter for noisy photos and the noise training.

F. Choosing a filter for each noise type

Model CA3 first decides the noise type. To find salt-and-pepper noise it counts isolated points with the point mask that has 8 in the centre and -1 around it [15]. A pixel with a response above 600 counts as a lonely dot. CA3 then uses the filter with the best SSIM from Table VII. This point detector works poorly once the photo is resized (Section V-G), so the final pipeline does not use it. The corrected model CA3 v2 has a lower validation score than CA and falls to 0.76 at σ 25. The filter that gives the best-looking image is not always the best filter for the classifier.

TABLE XI
MACRO-F1 ON DEEPQUESTAI PHOTOS WITH ADDED DAMAGE (S&P = SALT-AND-PEPPER NOISE)

Model	Clean	Dark	Blur	JPEG	Noise σ 5	Noise σ 10	Noise σ 15	Noise σ 25	S&P 2%	S&P 5%
CA (final)	**0.9002**	**0.8678**	**0.8887**	**0.8772**	**0.8937**	**0.8759**	**0.8599**	**0.8145**	**0.8632**	**0.8197**
CA2	0.9010	0.8772	0.8813	0.8868	0.8942	0.8843	0.8684	0.8365	0.8611	0.8187
CA3 v2	0.8912	0.8739	0.8809	0.8673	0.8870	0.8751	0.8560	0.7628	0.8408	0.7907
CN	0.8871	0.8838	0.8678	0.8612	0.8815	0.8698	0.8569	0.7983	0.8430	0.7879
CRN	0.8974	0.8776	0.8858	0.8859	0.8984	0.8876	0.8696	0.8058	0.6857	0.4407
CR	0.8995	0.8895	0.9062	0.8731	0.8791	0.7970	0.6614	0.3957	0.6364	0.4339

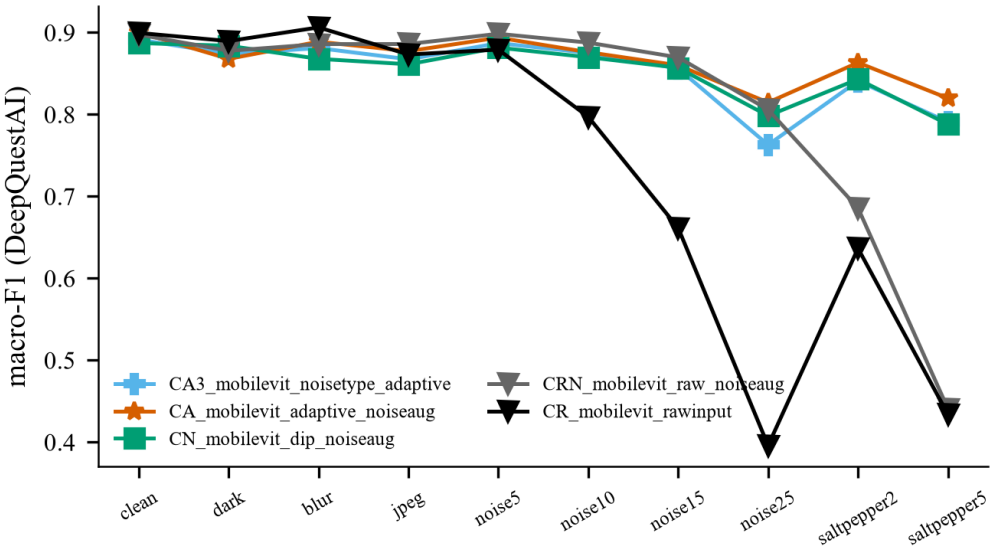


Fig. 25. Macro-F1 on DeepQuestAI under each kind of damage. CA2 was tested later in a separate run with the same damage, so it is in Table XI but not in this chart.

VII. SOFTWARE PACKAGE

The final model is released as a small Python package named firesmoke. It was built with PyTorch [17], timm [16] and OpenCV [29]. An export step packs the trained weights, the noise threshold, the filter settings and the class names into one file of about 8 MB. The package uses the same DIP code as the training, so a new photo is treated in the same way as the training photos.

The package can be used in three ways: from the command line, from Python, and from a Google Colab notebook that keeps the model and the results in Google Drive. For every photo it saves the label and the confidence, the measured σ, the decision (clean or noisy), an image of all steps, a short explanation in words and a file with all numbers. It also saves an image that compares the input with the output of the DIP step, with the PSNR, the SSIM and the share of changed pixels. Fig. 26 shows this image for a photo with added noise. For a clean photo the package reports a PSNR of 100 dB, an SSIM of 1.000 and 0% changed pixels, which shows that clean photos are not touched.

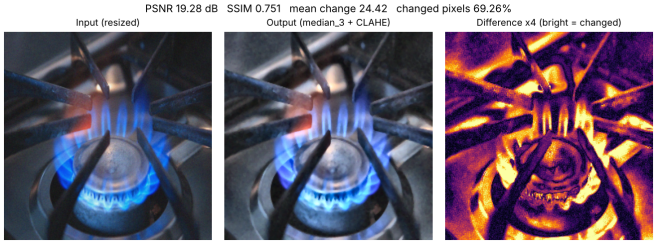


Fig. 26. Input and output comparison saved by the package for a photo with added Gaussian noise. Bright areas in the difference image are the pixels that changed.

On the laptop GPU, the model needs about 4.6 ms per photo on the validation set with a batch of 32. In one test on a new photo, a gas stove with a blue flame was called normal with a confidence of 96.1%. Its noise level was 0.89, so it was not filtered. This answer is useful for an alarm system, because a cooking flame is not a danger. It also shows a limit: the model learned fire as orange and yellow flames.

VIII. DISCUSSION

The results agree with DIP theory [15]. The median filter is an order-statistic filter. It removes single wrong dots and keeps edges. This is why it is the best filter for salt-and-pepper noise,

and why CA does not break on this noise while CR and CRN do.

Every filter also removes some real detail. The median filter alone changes a photo to a PSNR of about 32 dB. The adaptive rule avoids this loss on clean photos. This is why CA is better than C and CN on clean photos.

Global histogram equalisation can stretch flat areas too much and make noise stronger. CLAHE works on small tiles and limits the stretch with a clip limit. Using only the L channel keeps the hue, so the fire colour stays correct.

Area interpolation acts as a low-pass filter. It lowers Gaussian noise and shrinks salt-and-pepper dots. This explains why the isolated point count stopped working when the noise was added before resizing.

The smoke mask is weak because smoke is grey. Sky, fog, roads and walls have the same colour. A colour rule alone cannot separate them. The trained network does much better, because it also uses shape and texture.

Filtering does not remove all noise. Noise training teaches the network to ignore the rest. DIP and deep learning work best together.

IX. Limitations and Future Work

- Each model was trained only once, with one random seed. Small differences between models may be due to chance.
- The first filter study added noise after resizing. We corrected it and trained CA3 again. Both results are reported.
- The smoke mask marks about 19% of the area of normal photos.
- About 12% of the normal DeepQuestAI photos are still called fire or smoke.
- The brightness rule (CA2) helped only a little on dark photos (0.877 against 0.868 for CA), and it marked 22% of ordinary photos as dark.
- The result folder of model CA2 was deleted by mistake. We tested its saved model again without training and rebuilt its results. Its validation score matched the original run exactly (0.9078).
- The system uses single photos. It does not use motion, which is a strong sign of smoke in video.
- The model learned fire as orange and yellow flames. Blue flames may be missed.

In future work we plan to train every model with several seeds and use paired tests, to add video and motion features, to improve the smoke mask with texture, and to run the package on a small edge device with a camera.

X. Conclusion

This work joined classic digital image processing with a small vision transformer for fire and smoke classification. The system measures the noise in each photo and filters only the noisy ones. The final model has the best validation macro-F1 of 16 classifiers (0.917), reaches 0.900 on a test set from a new source and gives false alarms on only 2.2% of look-alike photos. Under added noise, blur, darkness and JPEG damage it stays at 0.81 or higher, while the same network on raw photos falls to 0.40. The system is available as a Python package that shows every intermediate step.

References

[1] K. Muhammad, J. Ahmad, and S. W. Baik, "Early fire detection using convolutional neural networks during surveillance for effective disaster management," Neurocomputing, vol. 288, pp. 30–42, 2018.

[2] S. Dodge and L. Karam, "Understanding how image quality affects deep neural networks," in Proc. 8th Int. Conf. Quality of Multimedia Experience (QoMEX), 2016, pp. 1–6.

[3] D. Hendrycks and T. Dietterich, "Benchmarking neural network robustness to common corruptions and perturbations," in Proc. Int. Conf. Learn. Represent. (ICLR), 2019.

[4] T. S. Huang, G. J. Yang, and G. Y. Tang, "A fast two-dimensional median filtering algorithm," IEEE Trans. Acoust., Speech, Signal Process., vol. 27, no. 1, pp. 13–18, 1979.

[5] K. Zuiderveld, "Contrast limited adaptive histogram equalization," in Graphics Gems IV, P. S. Heckbert, Ed. San Diego, CA, USA: Academic Press, 1994, pp. 474–485.

[6] J. Immerkær, "Fast noise variance estimation," Comput. Vis. Image Understand., vol. 64, no. 2, pp. 300–302, 1996, doi: 10.1006/cviu.1996.0060.

[7] S. Mehta and M. Rastegari, "MobileViT: Light-weight, general-purpose, and mobile-friendly vision transformer," in Proc. Int. Conf. Learn. Represent. (ICLR), 2022.

[8] T.-H. Chen, P.-H. Wu, and Y.-C. Chiou, "An early fire-detection method based on image processing," in Proc. IEEE Int. Conf. Image Process. (ICIP), 2004, pp. 1707–1710.

[9] B. U. Töreyin, Y. Dedeoğlu, U. Güdükbay, and A. E. Çetin, "Computer vision based method for real-time fire and flame detection," Pattern Recognit. Lett., vol. 27, no. 1, pp. 49–58, 2006.

[10] T. Çelik and H. Demirel, "Fire detection in video sequences using a generic color model," Fire Safety J., vol. 44, no. 2, pp. 147–158, 2009, doi: 10.1016/j.firesaf.2008.05.005.

[11] P. V. A. B. de Venâncio, A. C. Lisboa, and A. V. Barbosa, "An automatic fire detection system based on deep convolutional neural networks for low-power, resource-constrained devices," Neural Comput. Appl., vol. 34, pp. 15349–15368, 2022, doi: 10.1007/s00521-022-07467-z.

[12] C. Catargiu, N. Cleju, and I. B. Ciocoiu, "A comparative performance evaluation of YOLO-type detectors on a new open fire and smoke dataset," Sensors, vol. 24, no. 17, Art. no. 5597, 2024, doi: 10.3390/s24175597.

[13] A. Howard et al., "Searching for MobileNetV3," in Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV), 2019, pp. 1314–1324.

[14] DeepQuestAI, "Fire-Smoke-Dataset," GitHub repository. [Online]. Available: https://github.com/DeepQuestAI/Fire-Smoke-Dataset (accessed Nov. 2026).

[15] R. C. Gonzalez and R. E. Woods, Digital Image Processing, 4th ed. New York, NY, USA: Pearson, 2018.

[16] R. Wightman, "PyTorch image models," GitHub repository, 2019, doi: 10.5281/zenodo.4414861.

[17] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in Adv. Neural Inf. Process. Syst., vol. 32, 2019, pp. 8024–8035.

[18] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in Proc. Int. Conf. Learn. Represent. (ICLR), 2019.

[19] K. He, J. Sun, and X. Tang, "Single image haze removal using dark channel prior," IEEE Trans. Pattern Anal. Mach. Intell., vol. 33, no. 12, pp. 2341–2353, 2011.

[20] N. Otsu, "A threshold selection method from gray-level histograms," IEEE Trans. Syst., Man, Cybern., vol. 9, no. 1, pp. 62–66, 1979.

[21] T. Ojala, M. Pietikäinen, and T. Mäenpää, "Multiresolution gray-scale and rotation invariant texture classification with local binary patterns," IEEE Trans. Pattern Anal. Mach. Intell., vol. 24, no. 7, pp. 971–987, 2002.

[22] R. M. Haralick, K. Shanmugam, and I. Dinstein, "Textural features for

image classification," IEEE Trans. Syst., Man, Cybern., vol. SMC-3, no. 6, pp. 610–621, 1973.

[23] C. Cortes and V. Vapnik, "Support-vector networks," Mach. Learn., vol. 20, no. 3, pp. 273–297, 1995.

[24] L. Breiman, "Random forests," Mach. Learn., vol. 45, no. 1, pp. 5–32, 2001.

[25] B. Efron and R. J. Tibshirani, An Introduction to the Bootstrap. New York, NY, USA: Chapman & Hall, 1993.

[26] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," IEEE Trans. Image Process., vol. 13, no. 4, pp. 600–612, 2004.

[27] C. Tomasi and R. Manduchi, "Bilateral filtering for gray and color images," in Proc. IEEE Int. Conf. Comput. Vis. (ICCV), 1998, pp. 839–846.

[28] A. Buades, B. Coll, and J.-M. Morel, "A non-local algorithm for image denoising," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2005, pp. 60–65.

[29] G. Bradski, "The OpenCV library," Dr. Dobb's J. Softw. Tools, 2000.